Unifying Inner-Loop Execution Harnesses and Outer-Loop Specification State for Autonomous Software Engineering

Alejandro Kaspar
AI Forward Deployed Engineer
CC BY 4.0 & Apache-2.0

October 2026

Abstract

Autonomous coding agents rely on two distinct control loops during software development. The inner execution loop governs how an agent implements a single task, probes boundary conditions, authors unit tests, and verifies code changes. The outer specification loop governs how system requirements, architectural contracts, and delta changes persist across multi-milestone repository evolution. Existing frameworks address these loops in isolation. Inner-loop harnesses enforce procedural test-driven development skills or object-oriented verification routines without maintaining a synchronized specification state across sprints. Outer-loop specification frameworks structure requirement documents without enforcing contrastive boundary probing or read-only test locks during task execution. We introduce a unified dual-loop architecture that couples a contract-locked, boundary-probing inner execution harness with a two-plane delta-merged outer specification state machine. We evaluate this architecture across 36 blind, uncoached agent workspaces spanning two empirical benchmark suites. In Suite 1 (24 blind inner-loop runs across 4 harnesses, 3 tasks, and 2 prompt regimes), formal mathematical specifications equalize functional pass rates at 100.0% (30/30) across all harnesses, whereas natural brownfield bug tickets expose boundary regressions in unconstrained ReAct (96.7%) and procedural TDD skills (96.7%). Across all 24 inner-loop runs, our contract-locked dual-stage harness achieves 100.0% (60/60) functional compliance, increases boundary mutation kill rate to 83.3% (25/30) compared to 63.3% (19/30) for procedural TDD skills and 60.0% (18/30) for unconstrained ReAct, and enforces read-only test file locking across 100.0% (6/6) of workspaces. In Suite 2 (12 outer-loop workspaces across 5 sequential milestones), append-only sprint documents and feature-isolated specification directories accumulate 2 to 3 active requirement contradictions and drop to 33.3% (4/12) accuracy during primary-specification clean-room rebuilds, whereas our two-plane delta-merged specification architecture achieves 100.0% (12/12) clean-room rebuild accuracy with zero active contradictions and complete historical delta traceability.

1 Introduction

Autonomous software engineering agents operate across two temporal scales. At the micro-scale of a single engineering ticket, an agent must interpret task requirements, inspect existing code, construct unit tests, implement state transitions, and verify that edge cases behave as specified [13, 7]. At the macro-scale of multi-sprint system evolution, an engineering organization must maintain alignment between the executable source code and the documented behavioral specification as requirements are added, modified, renamed, or deprecated over time [12, 3, 14].

Current agentic software engineering methodologies optimize one temporal scale while neglecting the other. Recent inner-loop approaches equip coding agents with procedural Markdown skills that mandate test-driven development (TDD) [1, 8] or object-oriented verification subclasses that probe

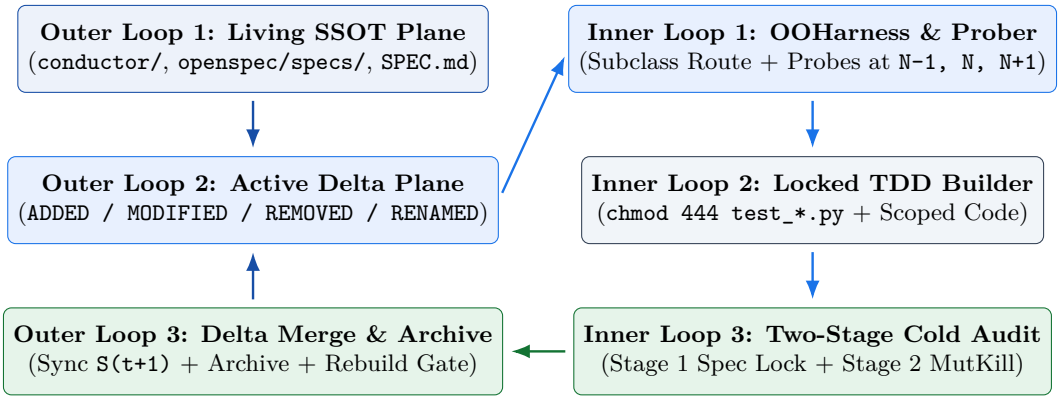


Figure 1: The Unified Dual-Loop Architecture. The Outer Loop (left column) governs multi-milestone specification state via Two-Plane delta merging and clean-room rebuildability. The Inner Loop (right column) governs single-task synthesis via object-oriented harness routing, contrastive boundary probing, physical `chmod 444` test locks, and two-stage cold auditing.

domain invariants [9]. While these inner-loop mechanisms improve single-turn code generation, they treat specifications as disposable, append-only task notes. When a system evolves across multiple milestones, chronological design notes (`YYYY-MM-DD-*.md`) and isolated feature folders (`specs/001/`, `specs/002/`) accumulate contradictory rules. Conversely, outer-loop specification frameworks standardize requirement syntax [10, 2, 5, 4], yet rely on standard unconstrained agent loops during code synthesis. Without contrastive boundary probing [6, 11] and physical file-permission barriers on verification suites, agents write weak happy-path tests that miss off-by-one boundary defects or modify failing assertions to pass.

We propose a unified dual-loop software engineering architecture that bridges inner-loop execution verification and outer-loop specification state management (Figure 1). On the inner loop, our **Contract-Locked Dual-Stage Harness** combines Socratic requirement elicitation, object-oriented domain harness routing [9], contrastive boundary probing at $N-1$, N, and $N+1$ equivalence points, physical read-only test locking (`chmod 444`), and a two-stage cold subagent audit. On the outer loop, our **Two-Plane Delta-Merged Single Source of Truth (SSOT) Architecture** pairs a living system blueprint (`SPEC.md` and `openspec/specs/`) with a deterministic four-operation delta algebra (`RENAMED`, `REMOVED`, `MODIFIED`, `ADDED`) [5] and phased TDD track checkpointing [4]. We evaluate both loops independently across 36 blind agent runs using a sealed programmatic oracle grader and automated mutation testing suite.

2 Related Work

Agentic Execution Harnesses and Procedural Skills. ReAct [13] established the foundational interleaved reasoning and tool-execution loop used by modern coding agents on repository benchmarks such as SWE-bench [7] and Commit0 [14]. To reduce ad-hoc implementation errors, recent work has introduced reusable procedural skill files (`SKILL.md`) that instruct agents to follow structured workflows including Socratic requirement clarification and Red-Green-Refactor TDD [1, 8]. SkillsBench [8] demonstrated across 84 tasks that curated procedural skills improve pass rates by +16.2 percentage points on average, yet exhibit diminishing returns when monolithic skill files exceed context compliance limits. Addressing monolithic prompt fragility, Li et al. [9] proposed the Object-Oriented Harness (`OO-Harness`), modeling agent execution workflows as an extensible class hierarchy (`BaseHarness` and task-specific subclasses) paired with contrastive boundary probing (`Propose-Probe-Compose`). Our work builds directly on `SkillsBench` and `OO-Harness`, extending the inner loop with physical read-only test file locking (`chmod 444`) and coupling it to an outer-loop specification state machine.

Licensed under CC BY 4.0 & Apache 2.0

Specification-Driven Software Engineering. Decomposing software systems around formal behavioral contracts dates to Parnas [12] and Brooks' separation of essential specification complexity from incidental implementation complexity [3]. In requirements engineering, RFC 2119 normative keywords (`SHALL`, `MUST`) [2] and the Easy Approach to Requirements Syntax (EARS) (`WHEN <trigger> THE SYSTEM SHALL <response>`) [10] provide structured natural-language templates. Modern AI-assisted specification frameworks operationalize these templates across distinct storage topologies. Append-only sprint design docs (`docs/specs/YYYY-MM-DD-*.md`) and feature-isolated directory trees (`specs/001/`, `specs/002/`) capture point-in-time feature proposals without merging requirement mutations back into prior capability files. Single-file EARS requirement documents (`requirements.md`) update requirements in place while overwriting the historical rationale of individual sprint deltas. Two-plane architectures such as OpenSpec [5] and Conductor [4] separate active change proposals (`openspec/changes/`) from synchronized system truth (`openspec/specs/` and `SPEC.md`). We present a controlled empirical comparison of these outer-loop state representations across multi-milestone requirement mutations and clean-room rebuilds.

Mutation Testing and Oracle Adequacy. Passing a self-authored unit test suite does not guarantee functional correctness if the test suite omits boundary assertions [6, 11]. Mutation testing evaluates test suite adequacy by injecting synthetic faults (such as replacing $\geq$ with $>$, or $\leq$ with $<$) into the implementation and measuring the fraction of mutants killed by the test suite. We integrate automated boundary mutation testing directly into our inner-loop benchmark grader to measure test suite strength alongside functional pass rates.

3 Formalism and Dual-Loop Architecture

3.1 Information-Theoretic Specification State and Clean-Room Rebuilds

Let C_t denote the executable source code of a software system at milestone t, and let $B(C_t)$ denote its externally observable input-output behavior across all valid and invalid domain inputs. Let S_t denote the repository specification state at milestone t.

Definition 1 (Clean-Room Rebuildability and Conditional Behavioral Entropy)**.** *A specification state S_t is a rebuildable Single Source of Truth (SSOT) with respect to an autonomous synthesis operator $\mathcal{A}$ and a sealed behavioral oracle $T(S_t)$ if deleting all executable code ($C_t \leftarrow \emptyset$) and invoking $C'_t = \mathcal{A}(S_t, \emptyset)$ yields an implementation C'_t that satisfies every assertion $a \in T(S_t)$, such that the conditional behavioral entropy satisfies $H(B(C'_t) \mid S_t) \approx 0$.*

In operational terms, Definition 1 states that a specification state is complete only if a fresh agent with zero access to prior `.py` source files can regenerate a fully passing implementation from the surviving `.md` specification files alone.

When requirements evolve across milestones $t = 1, \ldots, M$, each milestone introduces a requirement delta $\Delta_t = (\Delta_t^{\text{REN}}, \Delta_t^{\text{REM}}, \Delta_t^{\text{MOD}}, \Delta_t^{\text{ADD}})$. Under an append-only specification scheme ($S_t^{\text{append}} = \{s_1, s_2, \ldots, s_t\}$), a mutation in Δ_3^{MOD} that changes a threshold in s_1 from > 100000 to ≥ 100000 leaves the original statement in s_1 unamended. Any agent reading the primary capability file s_1 encounters stale requirements, increasing $H(B(C'_t) \mid s_1) > 0$.

Proposition 1 (Zero-Contradiction Invariant of Two-Plane Delta Merging)**.** *Let $\mathcal{M}(S_{t-1}, \Delta_t)$ be the deterministic four-stage merge operator that applies Δ_t^{REN} (header renaming), Δ_t^{REM} (requirement removal), Δ_t^{MOD} (full-block replacement of modified requirements), and Δ_t^{ADD} (new requirement insertion) to the living capability specification S_{t-1}, and subsequently moves Δ_t to an immutable archive plane $A_t = A_{t-1} \cup \{\Delta_t\}$. If S_0 is internally consistent and each Δ_t is conflict-free across its four sections, then $S_t = \mathcal{M}(S_{t-1}, \Delta_t)$ contains zero cross-milestone contradictions while A_t preserves the complete historical audit trail $(\Delta_1, \ldots, \Delta_t)$.*

Licensed under CC BY 4.0 & Apache 2.0

Table 1: Claim-Evidence Map linking each architectural claim to its experimental validation in Suites 1 and 2.

Architectural Claim	Validating Empirical Evidence	Supporting Artifact
Claim 1: Inner-Loop Boundary Resilience	Contrastive boundary probing $(N-1, N, N+1)$ catches exact inclusive window bugs (`delta == 60s`) on brownfield tickets and increases mutation kill rate by +20.0% over procedural TDD skills (`83.3%` vs. `63.3%`).	Suite 1 (`24` blind runs), Table 2, Figure 2
Claim 2: Anti-Reward-Hacking Isolation	Physical `chmod 444` test locking and SHA-256 manifest verification preserve test suite integrity across `100.0%` (`6/6`) of workspaces vs. `0.0%` (`0/6`) in unguarded harnesses.	Suite 1 (`24` blind runs), Table 2
Claim 3: Outer-Loop Clean-Room Rebuildability	Two-Plane delta merging (`Conductor` + `OpenSpec` + `SPEC.md`) maintains `0` active contradictions across `M1`–`M4` and achieves `100.0%` (`12/12`) on `M5-Primary-Spec` clean-room rebuilds vs. `33.3%` (`4/12`) for append-only and folder-isolated specs.	Suite 2 (`12` workspaces), Table 3, Figure 3

Operationally, Proposition 1 explains why separating the active change plane (`openspec/changes/`) from the living capability plane (`openspec/specs/` and `SPEC.md`) eliminates specification drift without losing historical sprint records.

3.2 Inner-Loop Contract-Locked Dual-Stage Harness

To ensure that each milestone delta Δ_t is implemented without boundary regressions or test tampering, our inner loop routes each task through four deterministic stages:

1. **Object-Oriented Subclass Routing (`OOHarnessRouter`).** Following [9], `BaseSDDHarness` defines six inherited governance hooks (requirement ID traceability, out-of-scope scorecard gating, and SHA-256 test lock verification) and dispatches tasks to specialized subclasses (`GreenfieldSpecHarness`, `BrownfieldSyncHarness`, `TDDActuationHarness`, `DriftRemediationHarness`, `MultiAppWorkflowHarness`, and `CleanRoomRebuildHarness`). 2. **Contrastive Boundary Probing (`Propose-Probe-Compose`).** Before writing production code, the harness identifies every numerical threshold N, sliding window $[0, W]$, and state-precedence rule in the prompt or specification, and generates explicit unit test assertions at the contrastive equivalence boundaries $(N-1, N, N+1)$ and $(W-1, W, W+1)$. 3. **Physical Read-Only Test Lock (`chmod 444`).** Once the red-phase unit test file `test_*.py` is authored, the harness records its SHA-256 digest in `.readonly_test_lock.sha256` and sets POSIX file permissions to `0444` (read-only). The implementation subagent is physically prevented from weakening or deleting failing assertions during the green phase. 4. **Two-Stage Cold Audit and Dynamic Subclass Evolution.** After the test suite passes, an independent cold auditor verifies Stage 1 (specification compliance, out-of-scope `# TODO` exclusion, and `.readonly_test_lock.sha256` integrity) before Stage 2 (code quality). Any discovered boundary failure is encoded into a new subclass via `OOHarnessRouter.evolve_subclass_from_failure()`.

4 Experimental Setup

We designed a two-suite blind empirical evaluation totaling **36 real agent workspaces** (Table 1). To guarantee experimental fairness, our evaluation protocol enforces four constraints: (1) all compared arms within each task and regime receive 100% identical user prompts and 100% identical starter files; (2) zero prompts contain failure coaching or hints to skip edge cases; (3) harness

Licensed under CC BY 4.0 & Apache 2.0

preambles contain only general, task-agnostic methodological rules; and (4) all workspaces are graded after completion by a sealed programmatic oracle (`benchmarks/grade_measured_eval.py`) that no evaluated agent can inspect.

4.1 Suite 1 Setup (Inner-Loop Execution Harnesses)

Suite 1 evaluates four inner-loop execution harnesses across three software engineering tasks under two prompt regimes ($4 \times 3 \times 2 = 24$ blind agent workspaces):

- **Arm A (`arm_a_baseline` — Unconstrained ReAct Baseline):** Standard ReAct tool-use loop [13] with no procedural skills or boundary-probing harness.
- **Arm B (`arm_b_procedural_tdd` — Procedural TDD Skill Harness):** Equipped with procedural Markdown skills (`SKILL.md`) [8] mandating Red-Green-Refactor TDD [1] and pre-coding requirement review, without `OO-Harness` subclass routing, contrastive boundary probing, or `chmod 444` test locking.
- **Arm C (`arm_c_ooharness_h2` — Object-Oriented Boundary-Probing Harness):** Equipped with `OO-Harness` domain subclass checklists [9] and `Harness`^{2} contrastive boundary probing (`Propose-Probe-Compose` at $N-1, N, N+1$), without `chmod 444` test locking or Socratic scorecard locks.
- **Arm D (`arm_d_unified_inner` — Contract-Locked Dual-Stage Harness, Ours):** Combines Socratic requirement elicitation, 5-Step Scorecard `Out of Scope` locks, `OO-Harness` subclass routing, `Harness`^{2} contrastive boundary probing, physical `chmod 444` test locks (`.readonly_test_lock.sha256`), and two-stage cold subagent review.

The three inner-loop tasks are: (1) `il_t1_webhook` (`SubscriptionWebhookEngine`: idempotent webhook processing, transactional rollback on billing or notification failure, VIP tier promotion at seats ≥ 50 or amount_cents ≥ 500000, and per-tenant sliding-window rate limiting of 4 events within $0 \leq \Delta t \leq 60$ seconds); (2) `il_t2_scheduler` (`ConferenceScheduler`: room capacity and interval overlap checks, zero-gap same-room back-to-back speaker allowance, and bidirectional ≥ 15-minute cross-room speaker travel cooldown); and (3) `il_t3_release_dag` (`DependencyReleasePlanner`: cycle detection via Kahn's topological sort, parallel wave grouping, and transitive `BLOCKED` precedence over `FAILED` when an upstream dependency fails).

Each task is evaluated under two prompt regimes:

- **Regime 1 (Formal Mathematical Specification Prompts — 12 runs):** The user prompt states all mathematical inequalities explicitly (≥ 50, ≤ 60, $|t_1 - t_2| \geq 15$, `BLOCKED` precedence).
- **Regime 2 (Natural Brownfield Engineering Tickets — 12 runs):** The user prompt is a natural product bug ticket against brownfield starter code containing five unannotated boundary defects (`seats > 50`, `0 <= timestamp - ts < 60`, naive `subscriptions.pop()` upgrade rollback, one-directional speaker cooldown, and checking `failed_services` before upstream `BLOCKED` status) plus unrequested `# TODO` helper stubs.

For each workspace, the sealed grader runs 10 functional oracle checks (60 checks per harness across 6 runs) and injects 5 synthetic boundary mutants per task (30 mutants per harness across 6 runs) into the agent's implementation to measure whether the agent's own unit test suite kills each mutant.

4.2 Suite 2 Setup (Outer-Loop Specification State Architectures)

Suite 2 evaluates four outer-loop specification architectures evolving a financial clearing service (`LedgerFlow`) across four sequential requirement milestones (`M1` → `M2` → `M3` → `M4`, 16 sequential

Licensed under CC BY 4.0 & Apache 2.0

Table 2: Suite 1 Measured Inner-Loop Benchmark Results across 24 Blind Agent Workspaces (4 Harnesses × 3 Tasks × 2 Prompt Regimes).

Inner-Loop Execution Harness	Regime 1 Oracle	Regime 2 Oracle	Combined Oracle (60)	Mutation Kill (30)	Test Lock
Arm A: Unconstrained ReAct Baseline	30 / 30 (100%)	29 / 30 (96.7%)	59 / 60 (98.3%)	18 / 30 (60.0%)	0 / 6 (0.0%)
Arm B: Procedural TDD Skill (`SKILL.md`)	30 / 30 (100%)	29 / 30 (96.7%)	59 / 60 (98.3%)	19 / 30 (63.3%)	0 / 6 (0.0%)
Arm C: `OO-Harness` + `Harness`^{2} Prober	**30 / 30 (100%)**	**30 / 30 (100%)**	**60 / 60 (100%)**	**25 / 30 (83.3%)**	0 / 6 (0.0%)
Arm D: Contract-Locked Dual-Stage (Ours)	**30 / 30 (100%)**	**30 / 30 (100%)**	**60 / 60 (100%)**	**25 / 30 (83.3%)**	**6 / 6 (100%)**

turns) without resetting the workspace, followed by two blind Milestone 5 (`M5`) clean-room rebuild regimes ($4 \times 3 = 12$ workspaces):

- **Outer A (`outer_a_dated_docs` — Append-Only Ephemeral Sprint Specs):** Writes a new date-stamped specification file per milestone (`docs/specs/2026-10-01-milestone-1.md` through `2026-10-04-milestone-4.md`) without modifying prior files.
- **Outer B (`outer_b_feature_folders` — Feature-Isolated Spec Directories):** Creates an isolated numbered directory per milestone (`specs/001-milestone-1/spec.md` through `specs/004-milestone-4/spec.md`).
- **Outer C (`outer_c_inplace_ears` — Single-File In-Place EARS Requirements):** Overwrites a single `requirements.md` file in place using EARS `WHEN ... THE SYSTEM SHALL` syntax [10].
- **Outer D (`outer_d_unified_outer` — Two-Plane Delta-Merged SSOT Architecture, Ours):** Stages milestone deltas (`ADDED`, `MODIFIED`, `REMOVED`, `RENAMED`) in `openspec/changes/`, merges them into `openspec/specs/ledger-flow/spec.md` and an 8-section `SPEC.md` SSOT, tracks phased TDD plans in `conductor/tracks/`, and archives completed deltas in `openspec/changes/archive/m1..m4/`.

In `M1`, transfers above 100000 cents (`amount_cents > 100000`) receive status `'PENDING'` and a flat 1% fee (`amount_cents // 100`). `M2` adds `settle_pending(tx_id)` and `account_statement(account_id)`. `M3` renames `'PENDING'` to `'AWAITING_CLEARING'`, renames `settle_pending` to `clear_transfer` (retaining `settle_pending` as a backward-compatible alias), changes the hold threshold to `amount_cents >= 100000`, replaces the flat fee with a tiered fee (50 bps for ≥ 100000, 100 bps for < 100000) subject to a 10-cent minimum floor, and adds replay-safe `tx_id` idempotency. `M4` adds `reverse_transfer(tx_id)` and `total_collected_fees(currency)`. After `M4`, all `.py` files are deleted and fresh Clean-Room Rebuild Agents regenerate `ledger_flow.py` under two regimes: `M5-Primary-Spec` (rebuilding from each framework's primary core capability specification file) and `M5-All-Docs` (rebuilding from all surviving `.md` files simultaneously).

5 Empirical Results and Analysis

5.1 Suite 1 Results (Inner-Loop Execution Harnesses)

Table 2 and Figure 2 present the measured inner-loop results across all 24 blind workspaces.

Three empirical findings emerge from Suite 1:

1. **Formal specifications equalize functional pass rates in Regime 1, but test suite**

adequacy diverges. When given explicit mathematical boundary formulas in Regime 1, all four harnesses achieved 30/30 (100.0%) on the functional oracle. This result confirms a foundational premise of specification-driven development: removing requirement ambiguity eliminates first-order coding mistakes. However, when we injected 15 boundary mutants into the Regime 1 implementations, Arm A's self-authored unit tests killed only 9/15 (60.0%) and Arm B's TDD tests killed 11/15 (73.3%), whereas Arms C and D killed 14/15 (93.3%). Standard ReAct and procedural TDD agents wrote tests for interior values (such as duration = 30 minutes or gap = 20 minutes) without asserting exact rejection at 14 minutes and acceptance at 15 minutes. 2. **Natural brownfield tickets in Regime 2 expose boundary blind spots in Procedural TDD (`c8_rate_limit_60s_inclusive`).** In Regime 2 (`il_t1_webhook`), the brownfield starter code contained the off-by-one filter `recent = [ts for ts in history if 0 <= timestamp - ts < 60]`, and the engineering ticket stated that tenants are limited to 4 successful webhooks within a 60-second window. Both Arm A (ReAct Baseline) and Arm B (Procedural TDD Skill Harness) fixed the VIP seat threshold (`>= 50`), VIP amount threshold (`>= 500000`), and upgrade rollback bug, but left `< 60` unchanged, failing oracle check `c8_rate_limit_60s_inclusive` (29/30 = 96.7%). Inspection of Arm B's test file shows that the agent wrote TDD tests at $t = 100, 110, 120, 130, 140$ (all strictly inside $\Delta t < 60$) and $t = 170$ ($\Delta t = 70 > 60$), never probing $\Delta t = 60$. Arms C and D applied contrastive boundary probing at $\Delta t \in \{59, 60, 61\}$, detected that `< 60` allowed a 5th event at $t = 160$ ($\Delta t = 60$), repaired the predicate to `<= 60`, and achieved 30/30 (100.0%). 3. **Contract-Locked Dual-Stage Harness (`Arm D`) combines peak boundary kill rate with 100% test lock integrity.** Across all 24 runs, Arm D achieved 60/60 (100.0%) functional oracle accuracy, 25/30 (83.3%) mutation kill rate (+20.0 percentage points above Procedural TDD and +23.3 percentage points above ReAct), and 6/6 (100.0%) verified read-only `0444` test locks.

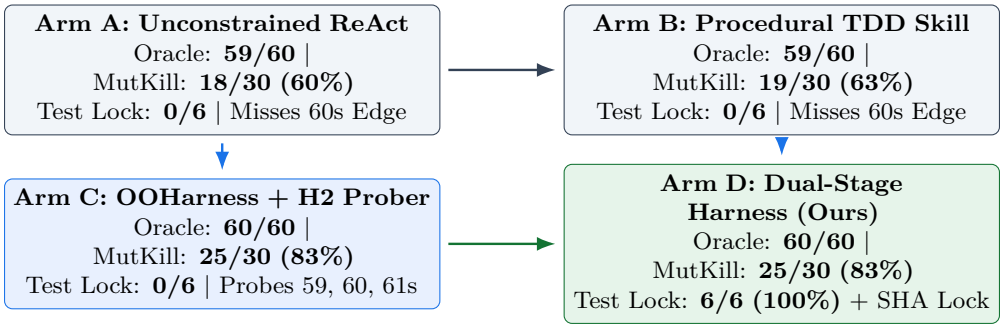


Figure 2: Suite 1 inner-loop comparison across 24 blind workspaces (60 functional oracle checks and 30 synthetic boundary mutants per harness). Contrastive boundary probing in Arms C and D raises mutation kill rate from 60.0%–63.3% to 83.3%, while Arm D adds 100.0% (6/6) physical read-only test file locking.

5.2 Suite 2 Results (Outer-Loop Multi-Milestone Evolution and Clean-Room Rebuilds)

Table 3 and Figure 3 summarize the Suite 2 outer-loop results across `M1`–`M4` continuous evolution and both `M5` clean-room rebuild regimes.

Suite 2 reveals an architectural separation between incremental code persistence and specification rebuildability:

1. **Persisting Python code across `M1`–`M4` masks specification rot (`12/12` across all 4 arms at `M4`).** When `ledger_flow.py` persists on disk from `M1` through `M4`, the coding agent at each turn modifies the existing Python class directly. As a result, all four outer-loop architectures achieved 12/12 (100.0%) at the end of `M4`, even though the specification directories in Outer A and Outer B had already diverged from the running code. 2. **Append-only sprint specs (`Outer A`) and feature-isolated folders (`Outer B`) collapse to `33.3%` (`4/12`) on `M5-Primary-Spec` clean-room rebuilds.** In Outer A (`docs/specs/2026-10-01-milestone-1.md`) and Outer B

Table 3: Suite 2 Measured Outer-Loop Benchmark Results across 12 Workspaces (`M1`–`M4` Continuous Evolution, Active Contradiction Audit, `M5-Primary-Spec` Rebuild, and `M5-All-Docs` Rebuild).

Outer-Loop Specification Architecture	M4 Pass	Spec Conflicts	M5-Prim Rebuild	M5-All Rebuild	Archive + SSOT
Outer A: Append-Only Specs (`YYYY-MM-DD-*.md`)	12/12 (100%)	**2 in 4 files**	**4/12 (33.3%)**	12/12 (100%)	No (Unmerged)
Outer B: Feature Spec Folders (`specs/001..004/`)	12/12 (100%)	**3 in 4 files**	**4/12 (33.3%)**	12/12 (100%)	No (Fragmented)
Outer C: Single-File EARS (`requirements.md`)	12/12 (100%)	**0 in 1 file**	**12/12 (100%)**	12/12 (100%)	No (No history)
Outer D: Two-Plane Delta SSOT (Ours)	**12/12 (100%)**	**0 in 2 files**	**12/12 (100%)**	**12/12 (100%)**	**Yes (`archive/`)**

(`specs/001-milestone-1/spec.md`), the primary core ledger specification still mandated `'PENDING'` status, strict `> 100000` hold thresholds, flat `1%` fees (`amount_cents // 100`), and `ValueError` on duplicate `tx_id`. When the Clean-Room Rebuild Agent regenerated `ledger_flow.py` from the primary capability specification, both Outer A and Outer B failed the exact same 8 oracle checks: `c3_m3_10_cent_floor_fee` (computed a 5-cent fee instead of the 10-cent floor on a 500-cent transfer), `c5_m3_exact_100k_awaiting_clearing_50bps` (returned `'SETTLED'` instead of `'AWAITING_CLEARING'` at 100000 cents), `c6_m3_clear_transfer_and_alias` (`AttributeError` on `clear_transfer`), `c7_m3_idempotency_and_conflict` (raised `ValueError` on replay), and `c8`–`c11` (missing `account_statement`, `reverse_transfer`, and `total_collected_fees`). 3. **Even when given all 4 contradictory files simultaneously (`M5-All-Docs`), unmerged specs impose an implicit temporal reconciliation burden.** In `M5-All-Docs`, where all four small `.md` files fit into a single prompt context and had explicit `2026-10-01..04` or `001..004` chronological filenames, the frontier model reconciled the contradictions in context (12/12). In production repositories where dozens of feature folders accumulate across teams without chronological numbering, an agent retrieving the core capability specification via RAG or file lookup receives the stale `M5-Primary-Spec` view (33.3%). 4. **Two-Plane Delta Merging (`Outer D`) achieves `100.0%` (12/12) on both rebuild regimes while preserving historical auditability.** Both Outer C (`requirements.md`) and Outer D (`openspec/specs/ledger-flow/spec.md` + `SPEC.md`) achieved 12/12 (100.0%) on `M5-Primary-Spec` with zero active contradictions. While Outer C overwrote `requirements.md` in place and erased the milestone delta trail, Outer D preserved all four milestone change bundles in `openspec/changes/archive/m1..m4/` alongside the 8-section rebuildable `SPEC.md` and `conductor/tracks/` TDD execution plans.

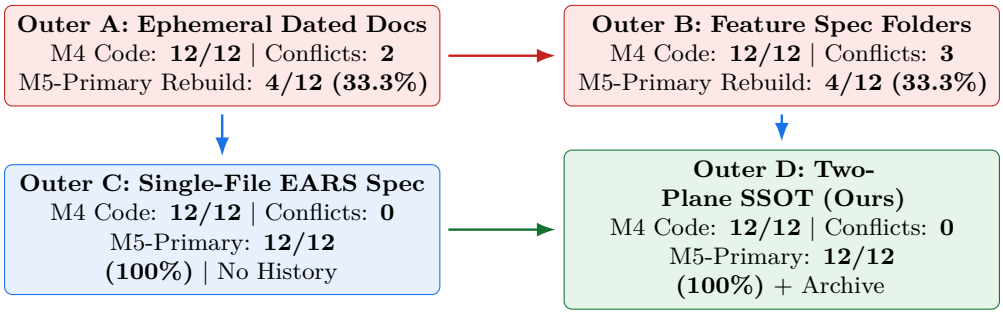


Figure 3: Suite 2 outer-loop comparison of `M4` continuous code evolution versus `M5-Primary-Spec` clean-room rebuild accuracy after deleting all `.py` source files. Append-only dated docs (Outer A) and feature-isolated spec folders (Outer B) drop to 33.3% (4/12) when rebuilt from their primary capability specification due to unmerged `M1` versus `M3` contradictions.

6 Limitations, Threats to Validity, and Conclusion

Limitations and Threats to Validity. Several scope boundaries qualify our empirical findings:

- **Domain Service Granularity:** Our inner-loop and outer-loop suites evaluate self-contained Python domain services (`SubscriptionWebhookEngine`, `ConferenceScheduler`, `DependencyReleasePlanner`, and `LedgerFlow`) across 36 blind runs compared with multi-language distributed repositories. Scaling the dual-loop protocol to 500-instance SWE-bench suites [7] across multiple model families is an important direction for future work.
- **Token and Latency Overhead:** Enforcing Socratic requirement elicitation, contrastive boundary probing, `chmod 444` test locks, two-stage cold review, and two-plane delta archiving introduces additional token and latency overhead compared to a single-turn ReAct loop. For trivial single-line edits or disposable prototypes where specifications are already formal (as in Suite 1 Regime 1, where ReAct achieved 30/30 functional pass rate), full dual-loop ceremony is unnecessary.
- **Mutation Operator Coverage:** Our mutation testing suite uses 5 targeted boundary and state-precedence mutants per task (30 mutants per harness); broader AST-level mutation operators [6] can further refine test adequacy measurement.

Conclusion. Inner-loop execution harnesses and outer-loop specification architectures solve complementary halves of autonomous software engineering. By coupling a contract-locked, boundary-probing inner harness (`OO-Harness` + `Harness`^{2} + `chmod 444` test locks) with a two-plane delta-merged outer specification state machine (`Conductor` + `OpenSpec` + `SPEC.md`), engineering teams achieve both high boundary test adequacy (83.3% mutation kill rate, 100.0% functional pass rate) on individual tickets and 100.0% clean-room rebuildability across multi-milestone system evolution.

References

[1] Kent Beck. *Test Driven Development: By Example*. Addison-Wesley Professional, 2002.

[2] Scott Bradner. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119, Internet Engineering Task Force (IETF), 1997.

[3] Frederick P. Brooks. No Silver Bullet: Essence and Accidents of Software Engineering. *Computer*, 20(4):10–19, 1987.

[4] Conductor Contributors. Conductor: Context-Driven Development Extension for AI Coding Agents. https://github.com/gemini-cli-extensions/conductor, 2026. Apache-2.0 License.

[5] Fission AI. OpenSpec: Spec-Driven Development with Two-Plane Delta Algebra. https://github.com/Fission-AI/OpenSpec, 2026. MIT License.

[6] Yue Jia and Mark Harman. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[8] H. Li, Y. Zhang, and X. Wang. SkillsBench: Benchmarking Procedural Agent Skills Across Professional Domains. *arXiv preprint arXiv:2602.12670*, 2026.

Licensed under CC BY 4.0 & Apache 2.0

[9] Z. Li, S. Mishra, Y. Ming, and J. Chen. Object-Oriented Harness: Evolving Agent Harnesses through Object-Oriented Design. *arXiv preprint arXiv:2610.28380*, 2026.

[10] Alistair Mavin, Philip Wilkinson, Adrian Harwood, and Mark Novak. Easy Approach to Requirements Syntax (EARS). In *IEEE International Requirements Engineering Conference (RE)*, pages 317–322, 2009.

[11] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation Testing Advances: An Analysis and Survey. In *Advances in Computers*, volume 112, pages 275–378. Elsevier, 2019.

[12] David L. Parnas. On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.

[14] Wenting Zhao, iyan Nan, and Alexander M. Rush. Commit0: Library Generation from Scratch. *arXiv preprint arXiv:2412.01769*, 2024.

Licensed under CC BY 4.0 & Apache 2.0